RMIT UNIVERSITY

School of Computing Technologies / STEM College

COSC2471 iPhone Software Engineering

Assessment 2: *Complete iOS SwiftUI App & Software Engineering (SE) Artefacts*

Due date:	Sunday, 4th October 2026, 11.59 pm AEST
COMPULSORY Face-to-face demo:	5th – 9th October 2026
Weighting:	100 marks (worth 40%)
Assessment type:	SwiftUI iOS app & SE artefacts
Group or individual assessment:	Group of 2

2.1 What is Assignment 2 about?

This is a group assessment. You must complete the tasks outlined **in a group of 2**.

As per the published COSC2471 class guide, this assignment requires a ***compulsory face-to-face demo***. Without the demo, **no marks** can be awarded. A booking schedule will be published closer to the demo date/week as indicated above. You must book and attend a 20-minute **in-person** demo. Both of you need to participate in the demo session; ***missing members will receive a zero for the assessment***.

For this assessment, you will develop a **fully functional iOS SwiftUI app** to a professional standard — that is, an app of a quality equivalent to one ready for submission to the App Store. (**You are NOT required to submit the app to the App Store.**)

☑ Note on your app idea: This assignment **does not** necessarily have to be an extension of the idea you chose for Assignment 1. If you wish to build on and extend your Assignment 1 concept, discuss this with your group partner first. Alternatively, you may choose an entirely new idea.

The final deliverable is an Xcode 26.* project for an iOS app, written in Swift and targeting iOS 26, along with the corresponding documentation. The use of storyboards is not permitted. Your client must be able to open and run the app on **either the iOS Simulator OR a physical iPhone**.

2.2 Extension requests and Late penalty

Please apply for an extension at least a day before the deadline and follow these guidelines:

- If you are seeking an extension of 7 days or less, please apply via the Assessments Extensions tool with the proper supporting documents. Do not email any staff member.

- If seeking an extension of >= 7 days, please submit your request online via the special consideration website (https://www.rmit.edu.au/students/my-course/assessment-results/special-consideration-extensions/special-consideration).

Late penalty

Assignments received late and without prior extension approval or special consideration will be penalised by a 10% deduction of the total possible score per calendar day late for that assessment.

For example, for this assessment, the late penalty per day will be 10 marks.

2.3 Use of Generative AI (IMPORTANT)

In this assessment, you may use AI only for **bounded process** support. This means AI may support your process, but it must not replace your own thinking, judgement, or authorship.

Within this process, you may use generative AI tools (for example ChatGPT, Claude, GitHub Copilot, or Xcode's predictive code completion) to:

- generate, complete or refactor Swift and SwiftUI code,
- explain compiler errors, Swift syntax or SwiftUI framework behaviour;
- write code comments and suggest sample data values.

Conditions: you remain fully responsible for all submitted code, must be able to explain any line of it if asked, and must include a brief AI Use Statement in your README naming the tools used and what they contributed

☑ You remain responsible for all ideas, decisions, reasoning, content and evidence submitted for marking. You must critically review, check, and adapt any AI output used during your process.

You must **submit the Bounded Process AI Declaration** with your assessment, **even if you did not use AI. If you did not use AI, you only need to complete the section confirming that no AI was used.**

If you used AI for approved process support, your declaration must identify the tool used, explain what it was used for, describe the AI output you received, explain what you accepted, rejected or revised, and describe how the AI interaction influenced your final submission.

Your **assessment submission is not complete unless it includes both the assessment work and the completed Bounded Process AI Declaration**. **Please refer to the exemplar in Appendix A to see how to complete the AI declaration.**

If you submit the assessment work **without the required declaration**, we will treat the whole assessment as **incomplete and not mark the submission**. The submission date will be the date you submit both the assessment work and the completed declaration.

2.4 Academic Integrity and Plagiarism

Academic integrity is about the honest presentation of your academic work. It means acknowledging others' work while developing your insights, knowledge, and ideas.

You should take extreme care to ensure that you have:

- acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarised, paraphrased, discussed, or mentioned in your assessment through the appropriate referencing methods.
- provided a reference list and /or bibliography of the publication details so your reader can locate the source if necessary. This includes material used from Internet sites &
- provided the prompts used to search for answers using Generative AI Tools.

Failing to acknowledge the sources of your material can lead to accusations of plagiarism, as it implies that you are presenting someone else's work or ideas as your own without proper referencing. RMIT University treats plagiarism as a serious offence and a form of misconduct. Plagiarism covers a variety of inappropriate behaviours, including:

- contract cheating- *paying someone else to do your work*
- failure to properly document a source
- copyright material from the internet or databases
- collusion between students
- unethical and unapproved use of Generative AI tools.

2.5 Tools

- Xcode 26.*
- Official GitHub organisation: https://github.com/rmit-iPSE-s2-2026
- Miro board
- DocC
- Text editor & PDF file creator
- iPhone (*only if demonstrated via an actual device*)

2.6 Course learning outcomes

This assessment is linked to the following course learning outcomes:

CLO 1	Design and develop mobile software applications using contemporary declarative frameworks and the iOS SDK across standalone enterprise and m-commerce contexts.
CLO 2	Analyse the iOS application development ecosystem and select appropriate platform components frameworks and services to support the development of AI-integrated mobile systems.
CLO3	Justify and apply relevant software design patterns and architectural approaches to structure maintainable, scalable and context-appropriate iOS applications.
CLO4	Critically analyse mobile app software engineering scenarios.
CLO5	Collaborate effectively in a development team to plan, build and communicate a mobile software product using appropriate software engineering processes.

2.7 Marking criteria

Kindly check the grading rubric for further details.

This assignment requires a ***compulsory face-to-face demo***. Without the demo, no marks can be awarded. A booking schedule will be published closer to the demo date/week as indicated above. You must book and attend a 20-minute in-person demo. Both of you need to participate in the demo session; ***missing members will receive a zero for the assessment***.

2.8 Consultations

You are encouraged to bring your questions to assessment-based consultations in the forthcoming weeks. No booking is required to attend.

2.9 Assessment Theme and Tasks →

RMIT UNIVERSITY

Theme

Is there an app you've always wanted to make but never had the time?

Here's your chance!

This is the assignment where your idea stops being a plan and becomes something real — something you can open on a phone, hand to someone, and say, "I built this." Done well, this becomes the centrepiece of your portfolio. Prospective employers are far more interested in an app they can run than in a transcript, and a polished, thoughtfully built iOS app tells them more about you as a developer than any interview question will.

So, aim high, and aim original. Don't rebuild a to-do list, a weather app, or a calculator clone. Bring us something that solves a problem you care about, serves a community you know, or approaches a familiar space from an angle nobody has tried. Novelty counts here — and so does taste. Note that Apple explicitly rejects apps that merely replicate what's already widely available on the App Store, so a derivative concept isn't just uninspiring; it's non-compliant.

Your job now is to develop an iOS SwiftUI app — complete with all the features— to a professional standard equivalent to an app ready for submission to the App Store. You are not required to submit it to the App Store.

How many features?

Draw on the knowledge you gained in Assessment 1 around designing for the iOS platform and building a prototype: your interface decisions, user flows, and Human Interface Guidelines: https://developer.apple.com/design/human-interface-guidelines

As a guide, aim for **five to six substantial features** — enough that the app does something genuinely useful, few enough that each one is finished to a professional standard.

Note that a **feature is not the same as a screen**. A single feature may span several screens: a booking feature, for example, might involve a search screen, a detail view, a confirmation step, and a history list. Count what the app does, not how many views you have built.

Whatever you choose, your features must collectively cover everything the assessment requires: local and cloud persistence, the REST API, your custom gesture, and the others listed below.

A polished app with five working features will always mark better than a sprawling one with ten half-built screens — if a feature doesn't earn its place, cut it.

IMPORTANT: What your app SHOULD NOT DO?

When developing an app concept for the iOS platform, even though you are not submitting the app via the App Store, you must ensure the idea does not fall foul of Apple's App Review Guidelines. The app must **not** contain offensive, discriminatory, or violent content, pornography, or anything that facilitates illegal activity such as the sale of controlled substances or unlicensed gambling. It must not risk physical harm to users, make unverifiable medical claims, or collect personal data without clear consent and a published privacy policy.

Finally, the app must **not** use private APIs, mine cryptocurrency on the device, or bypass Apple's in-app purchase system for digital goods. The full criteria are set out across the five sections of Apple's guidelines — Safety, Performance, Business, Design, and Legal — available at https://developer.apple.com/app-store/review/guidelines/

Please go to the next page for the assessment tasks →

RMIT UNIVERSITY

Part A (80 marks)

You are to make effective use of ***GitHub*** during the development process. Your repository must be a part of the **`rmit-iPSE-s2-2026`** organisation. Please read the rubric for further details. Your code repository must follow the format; for example, a valid repository name is `a2-s31111111-s3222222`.

Your project must use the **M-V-VM** approach with appropriate code separation. Each layer must contain only what belongs to it—data and its storage in the Model, presentation only in the View, and coordination between them in the ViewModel.

Misplaced code will cost marks **for each of the following requirements** even if the app runs correctly. Write efficient code; general code elegance rules apply. See the rubric for details.

Your task for this part is to complete all the features of your iOS app with the following requirements.

a) (10 marks) As a group, create and maintain one shared **Miro board** documenting the software-development process for your application. All group members must contribute to the board, which should demonstrate collaborative planning, task allocation, progress tracking, key decisions, and coordination throughout the project. This component assesses the effectiveness of your group's development process, not separate individual boards.

   Name the board using the format `a2-s31111111-s3222222` and share it with your lecturer at shekhar.kalra@rmit.edu.au. **Appendix B** provides an example of an appropriately structured Miro board for group software development.

   Complete the **App Concept section of the** PDF REPORT by filling in all required fields. Follow the questions provided in the template and include valid links to your GitHub repository and Miro board.

b) (15 marks = 5 + 10) Your app must now persist its data, using both **SwiftData** and the cloud database **Firebase** (**or any other approved cloud database**). Deciding what goes where is part of the task.

   Some data belongs on the device — settings, drafts, cached content, anything the user should still see with no signal.

   Other data belongs in the cloud — anything shared between users, synced across devices, or too valuable to lose if the phone is.

   Make a deliberate choice for each piece of data your app handles and be prepared to justify it in your PDF REPORT and during the demo.

c) (10 marks) Implement the networking approach you chose in your app to retrieve data from a remote, ***publicly available REST-based API***.

Place the REST API calling code separately from the Model, View, and ViewModel code files.

The REST API must make sense in the context of your iOS app. Specify the REST API's documentation page in your PDF REPORT.

d) (10 marks) Create a **complex *sequenced or simultaneous gesture*** suitable for your iOS app. This customised gesture must consist of ***multiple gestures***. DO NOT copy from the example covered during lectorial or practical; you will receive no marks for doing so.

You must create a new customised gesture and explain its role in enhancing user experience during a face-to-face demo.

e) (10 marks) Create 10-unit ***tests*** relevant to your codebase. Each of these tests must make sense, as you will be asked to explain them during the face-to-face demo.

f) (10 marks = 5 + 5) **Accessibility Implementation and UI Testing**. Review your application's three main screens and correct any accessibility problems. This includes adding meaningful labels to controls, ensuring elements are presented in a logical order to assistive technologies, supporting the largest Dynamic Type sizes without broken layouts, and providing appropriately sized interactive targets.

How you audit depends on how you'll demo your app.

- if you have a physical iPhone, enable VoiceOver and navigate each screen using VoiceOver alone, then check your layouts at the largest Dynamic Type setting. If you are working on the simulator, use Xcode's Accessibility Inspector to step through each screen's elements and run its Audit tab.

Specify a list of accessibility features in your PDF REPORT.

Implement **five UI tests** covering meaningful user journeys within your application. At least one of these tests must use `performAccessibilityAudit()` to automatically audit your interface. All tests must exercise substantial workflows, not trivial interactions.

g) (5 marks) Add a feature to your app that must be built with **UIKit**. Integrate UIKit and SwiftUI together. The feature must make sense in the context of your app. A simple implementation will receive low marks.

h) (10 marks) Create exhaustive code documentation of your project using Xcode's built-in documentation compiler, ***DocC***. You must include a *documentation catalog* to customise your documentation's landing page.

Please go to the next page for the remaining assessment tasks →

RMIT UNIVERSITY

Part B (20 marks)

☑ This is an advanced part and will require you to go beyond the course material and do your own research. You will not find direct answers in the course material, and as such, this requires significant effort on your part. DO NOT CUT-COPY-PASTE code generated by Agentic tools; **you will be asked to explain your approach, code and architecture** in the PDF REPORT and **during a face-to-face demo**.

i) (10 marks) Design and implement an **AI-agent** feature (*free-tier service; no paid AI service is to be used*) within your iOS SwiftUI application. The agent must be part of the application's runtime functionality.

Using an AI coding agent to develop the application does NOT satisfy this requirement. **A chatbot or generic conversational interface alone does NOT satisfy this requirement**.

Your agent must apply the **ReAct (Reasoning and Acting)** pattern: interpret a user's goal, select and invoke appropriate tools, observe the results, and determine the next action. It must:

1) complete a meaningful, multi-step user task;
2) use relevant user or application context; and
3) produce a structured outcome that users can review and edit.

Users must explicitly approve consequential actions. Your implementation must include input and output validation, error handling, privacy safeguards, and secure credential management.

j) (10 marks) Add ***Your Moment to Shine*** feature in the context of your iOS app that makes use of ***any ONE of the following advanced features***:

1. Core ML Integration – build a feature that applies on-device machine learning (e.g., voice classification, recommendation engine, or personalised predictions).

   Image classification does NOT count for this part.

2. Security & Privacy Enhancement – add advanced security features such as encrypted data storage or secure sharing mechanisms.

👉 You must choose **ONE** of the above options but the expectation is that your work is substantial—not something copied from the web or code-generated by GenAI. Treat this as a chance to stand out in your portfolio. This is ***your opportunity to shine and showcase your ability*** as an iOS developer. A strong effort here will not only build your technical confidence but also give you a project you can proudly feature in your portfolio and present to prospective employers.

RMIT UNIVERSITY

2.10 Submission instructions

Submit **1 PDF report** that contains all the requisite details mentioned above. Kindly make sure that your report contains

- completed AI declaration
- completed tasks and the following
    - URL of your GitHub Assignment 2 repository
    - URL of your shared Miro board.



APPENDIX A: How to fill in the AI Use Declaration properly

Read this before you type anything into AI declaration part the report template.

Most declarations I have seen during consultations looked like this:

> **Tool used:** Claude Code
>
> **Purpose of use:** To help with my code
>
> **What the AI produced:** Some suggestions
>
> **What I did with that output:** I changed it a bit
>
> **How it influenced the final submission:** All work is my own

That is a declaration that **declares nothing**. It tells your marker no more than a blank page would, and it is the single fastest way to make a marker doubt your authorship of work you may well have written entirely yourself.

The declaration is not a punishment, and it is not used to reward AI use. It is used to judge **whether you remained the author and the final decision-maker**. A student who used AI thoughtfully and says so precisely scores better on this than a student who used AI casually and hides it behind vague wording.

Before the examples: three ground rules

1. **You may expand the answer boxes.** The tables in the template are not fixed. Press Enter inside a cell and it grows. Add rows if you need them. Copy the whole five-field block if you used more than one tool. Nobody is marking you down for using space.
2. **But do not pad.** Aim for **three to six sentences per field** — roughly 60 to 120 words. One tool's complete five-field block should fit comfortably on a single page. If you used two or three tools for genuinely different purposes, that is normal. If you are filling in eight blocks, you are logging activity rather than reflecting on it, and the quality of the reflection will drop.
3. **Do not paste your prompt history.** We do not want a transcript. Quote **at most one or two prompts across the entire declaration**, and only where the wording of the prompt itself shows a decision you made — for example, where you deliberately asked for a hint rather than a solution. A wall of prompts is not evidence of thinking. It is evidence of scrolling.

> **A reminder on scope:** AI-bounded process use is permitted for **Part B only** in this assessment, and only for *process* support — brainstorming, alternative approaches, feedback on your reasoning, debugging hints, explanations of documentation. It is not permitted for generating the SwiftUI code you submit.

What each field is actually asking

Field 1 — Tool used

Weak — says nothing	Strong — shows your thinking
ChatGPT	ChatGPT (GPT-5, browser version), used 12–14 August 2026. I also used Xcode's built-in code completion, which is not prompt-based, but I am listing it for completeness.

Name the tool, the version if you know it, and roughly when. Thirty seconds of work, and it immediately signals you are taking the form seriously.

Field 2 — Purpose of use

This field asks what **process task** you used it for, and it should tie to a specific Part B task.

Weak — says nothing	Strong — shows your thinking
To help me with my layouts.	Debugging hints for **Task 2 (Layouts)**. My recipe detail screen was clipping the ingredient list at Dynamic Type size AX3 on the iPhone SE simulator. I asked for hints about *why* a fixed frame height interacts badly with large text — deliberately not asking for corrected code, since the brief permits process support only.

Notice what the strong version does: it names the task, names the symptom, names the device and text size, and states the boundary the student chose to work inside. A marker reading that knows exactly what happened.

Field 3 — What the AI produced

Describe the output honestly, including its limits. If it gave you something wrong or unusable, **say so** — that is some of the most credible content you can put in this form.

Weak — says nothing	Strong — shows your thinking
It explained the problem and gave me a fix.	A short prose explanation that `.frame(height:)` pins a container's height regardless of the text inside it, plus three general directions: use a flexible minimum height, let the text view size itself, or wrap the content in a scroll view. No code, because I had asked for prose only. It did not identify the actual cause of my bug.

Field 4 — What I did with that output

This is the field that carries the most marks and the one students most often waste. It asks what you *accepted, rejected, revised, or rewrote — and why*.

Weak — says nothing	Strong — shows your thinking
I used the suggestion and modified it to fit my app.	I rejected the scroll-view suggestion, because that card sits inside a screen that already scrolls and nesting them would have broken the swipe gesture on my step list. I tried the flexible-minimum-height approach and tested it on iPhone SE, 15 Pro and 16 Pro Max at default and AX3 sizes. It improved the clipping but did not remove it. I then traced the real cause myself in the view debugger: a hard-coded vertical spacing constant inside my custom layout's `placeSubviews`, which the AI had not mentioned and could not have known about. I replaced that constant with a value derived from the proposed size.

That paragraph is 100 words and it does everything the marker needs: a rejection with a design reason, a test method, an honest partial result, and independent diagnosis of something the AI got wrong.

Field 5 — How it influenced the final submission

Be specific about **which files or screens**, and be equally specific about what was untouched.

Weak — says nothing	Strong — shows your thinking
It influenced my code a bit but the final submission is mine.	Task 2 only, and within it two files: `RecipeDetailView.swift` and `IngredientCard.swift`. The influence was conceptual — it named the interaction between fixed heights and Dynamic Type, which I had not understood. Every modifier, value and the spacing logic in my custom layout are my own, and no AI output appears verbatim in the repository. Tasks 1, 3, 4 and 5 were completed without AI assistance.

If you went outside the permitted scope

If you pasted in AI-generated SwiftUI and kept it, **declare it accurately**. State what you took, where it is, and how much of it survived your editing. An honest declaration of over-use is assessed on its merits and may cost you marks on the relevant task. An **undeclared** one is an academic integrity matter, which is a substantially worse outcome. The form asks for an honest account, and it is written on the assumption that you will give one.

Signs of lip service (what makes a marker suspicious)

- No task, file or screen is ever named.
- Nothing is ever rejected. Every AI suggestion was apparently perfect.
- The verbs are all passive and agentless: "suggestions were provided", "the code was modified".
- Blanket claims — "all work is my own" — sitting next to five empty specifics.
- All five fields say roughly the same sentence in different words.
- The account is long but consists entirely of prompts, with no reasoning between them.

Quality of reflection, not length. Two specific paragraphs beat two vague pages.

SAMPLE MIRO BOARD STRUCTURE / GROUP ASSIGNMENT 2 iPSE APP CREATION JOURNEY

How your team got from idea to working app

A clean Miro board should reveal collaboration, decisions, progress and learning - not just the final product.

TEAM [names] **TIMEFRAME** [start date - submission date] **LAST UPDATED** [date]

BOARD NAME
a2-s31111111-s3222222
Editable sample - adapt to your team

01 PROJECT NAME

PURPOSE [One sentence: the problem your app solves]

USERS [Who you are designing for]

SUCCESS [Two signs that the project worked]

DEFINITION OF DONE [What the whole group agrees must be complete]

02 TEAM AGREEMENT & OWNERSHIP

WHO DOES WHAT [Name - primary responsibility]

WORKING RHYTHM [check-in days] | [main channel]

03 DISCOVERY TO WIREFRAMES

Show evolution, not only polished screens. Add short notes explaining what changed and why.

EARLY SKETCH	FEEDBACK	REFINED FLOW
[insert frame / link]	*[insert frame / link]*	*[insert frame / link]*

Capture: user flow | feedback received | design change | reason

04 KANBAN SNAPSHOT

Keep ownership, dates and movement visible. Link cards to GitHub issues or pull requests where useful.

BACKLOG	IN PROGRESS	REVIEW	DONE
[Task / outcome] Owner: [name] Due: [date] Evidence: [link]	**[Task / outcome]** Owner: [name] Due: [date] Evidence: [link]	**[Task / outcome]** Owner: [name] Due: [date] Evidence: [link]	**[Task / outcome]** Owner: [name] Due: [date] Evidence: [link]

05 DECISIONS, TENSION & LEARNING

DECISION
[What was decided + why]

DISAGREEMENT
[Options considered + how it was resolved]

BLOCKER / LEARNING
[Where you got stuck - what you tried - outcome]

06 JOURNEY TIMELINE

Mark the actual dates, turning points and scope changes. Keep the history visible.

ALIGN	DESIGN	BUILD	VERIFY	DELIVER
[dates] [milestone / evidence]	*[dates]* [milestone / evidence]	*[dates]* [milestone / evidence]	*[dates]* [milestone / evidence]	*[dates]* [milestone / evidence]

Keep it authentic and lightweight: use dates, owners, screenshots, links and short reflections. Do not tidy away disagreements, dead ends or changes of direction.

Sample only - your board should reflect your group's real journey.